



भारतीय प्रौद्योगिकी
संस्थान जम्मू
INDIAN INSTITUTE OF
TECHNOLOGY JAMMU



Automations & Workflows

with



n8n

By

Parmveer Nandal

Why Do We Need Automation?

Why Do We Need Automation?

Goal: Make data-driven decisions

Manual Decisions (Feeling/Intuition):

- Subjective and open to interpretation
- Difficult to estimate and justify ROI
- Often incurs wasted budgetary or time resources

Data-Driven Approach:

- Logic-driven and objective
- Improves reporting capacity and increases ROI visibility
- Requires fewer human, budgetary, and time resources

Automation is key to being
data-driven

Consequences of Manual Tasks

- Wasted time
- Human error from repeated low-value tasks (e.g., copying data)
- High human resource requirements
- Low employee happiness and retention

Benefits of Integrating Automation:

- Greater predictability and data availability
- Increased employee efficiency (focus on higher-value tasks)
- Higher ROI
- Lower needs for human resources

What is Automation?

Automation is the use of technology to perform tasks with minimal human intervention.

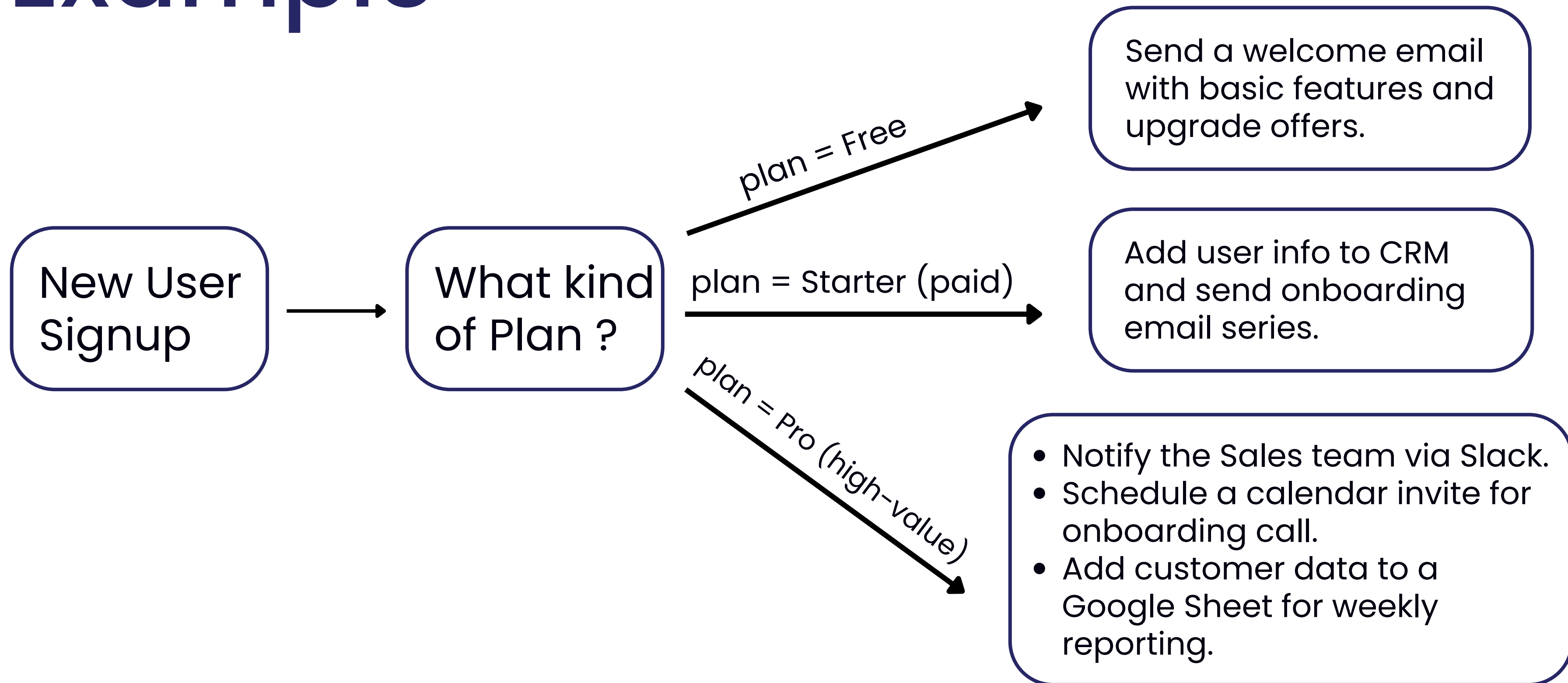
A predictable set of predetermined actions that transfers data from one point to another

What is Automation?

Everyday examples: Email filters, smart home routines, bank alerts

Benefits: Saves time, reduces errors, increases efficiency

Example



Core Concepts of Automation

needed to build workflows

Trigger

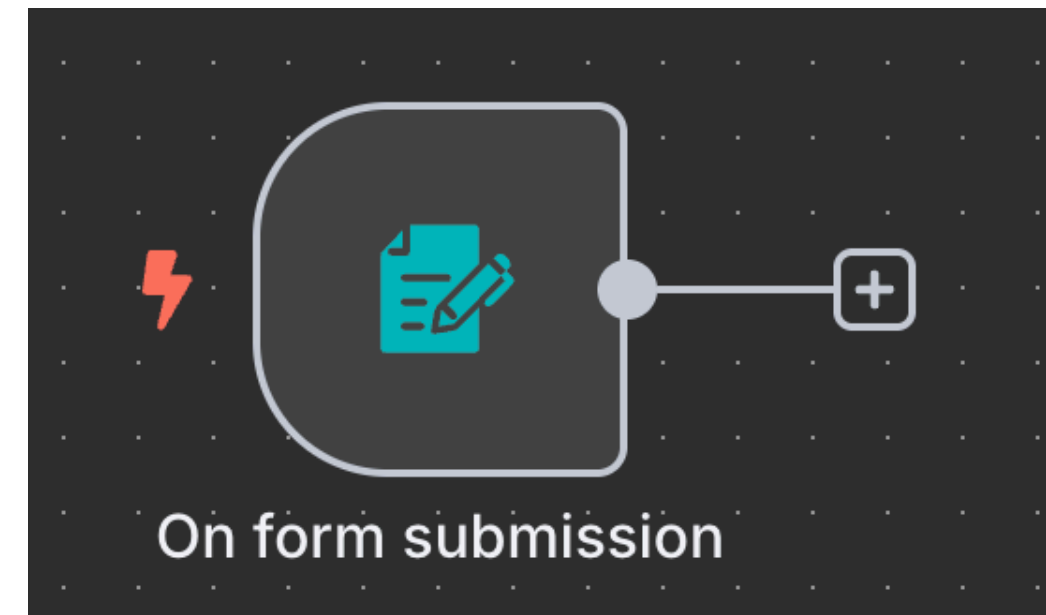
A trigger is what starts an automation.

- Manual

- Scheduled

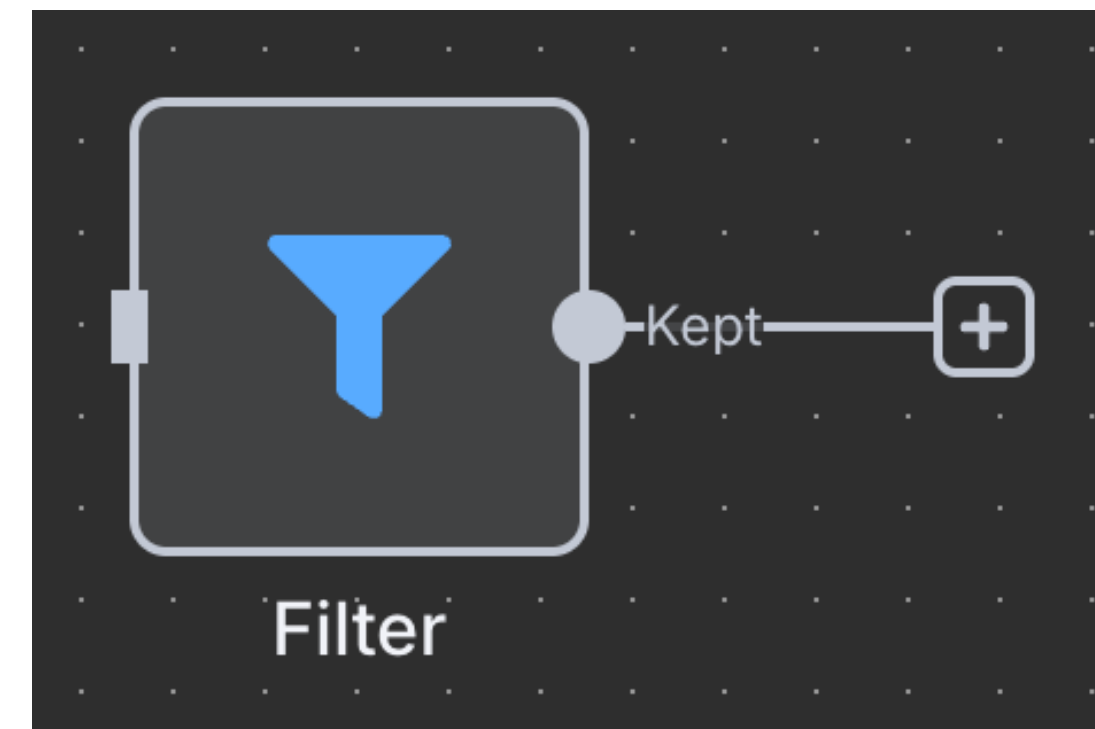
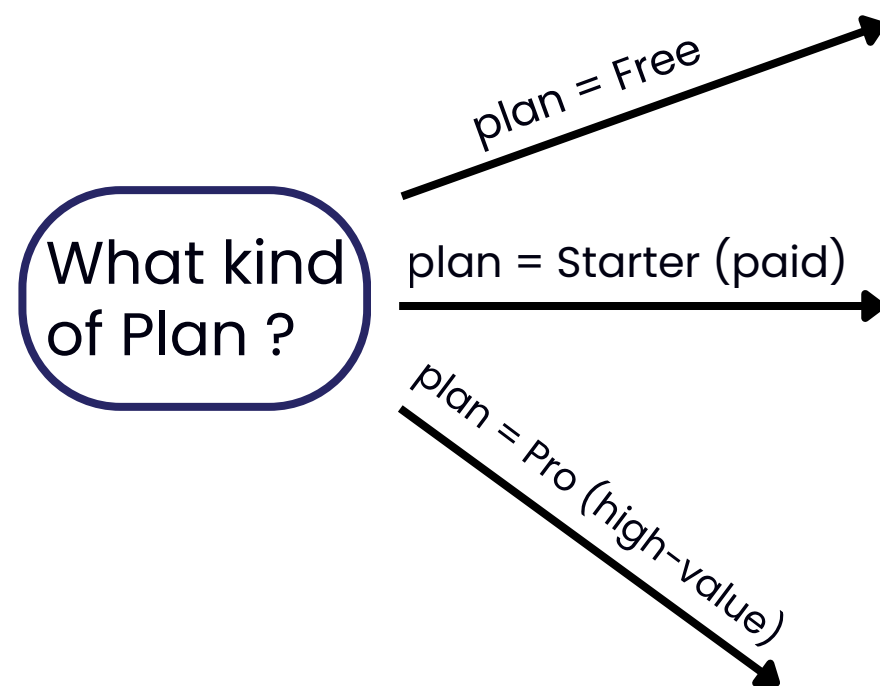
- Applications

- Webhook
- Property Update
- Form Submission



Filter

Allows or blocks certain data types from following specific paths based on conditions



Actions (Apps)

Actions allow us to interact with applications.

Action refers to a node that performs a task based on the data received from a Trigger or another node.

Actions (Apps)

Data Manipulation Actions

- Set – Set or update fields and values.
- Merge – Combine data from multiple nodes.
- Function – Write custom JavaScript to manipulate data.
- SplitInBatches – Loop through items in chunks.

Communication Actions

- Send Email – Send emails via SMTP, Gmail, Outlook, etc.
- Slack – Send messages to Slack channels/users.
- Telegram – Send messages via Telegram bot.
- Twilio – Send SMS or make voice calls.

CRM & Marketing Tools

- HubSpot / Zoho / Salesforce – Add/update contacts or deals.
- Mailchimp / ConvertKit / Sendinblue – Add to mailing list or campaign.

Database & Storage Actions

- Google Sheets – Create, read, update, delete rows.
- Airtable – Interact with Airtable bases.
- MySQL / PostgreSQL / MongoDB – Query or update databases.
- Notion – Add or retrieve content from Notion pages/databases.

Actions (Apps)

In n8n, Actions are the “doers” — they execute the operations once data flows into them. You can chain multiple actions in a single workflow.

Actions vary by application:

Dropbox: Upload a file, get a file, create a folder, move a file

Salesforce (CRM): Get a company, get a contact, create a lead, associate a lead to a company

Workflows

A workflow is a sequence of tasks that processes a set of data.

- Manual vs Automated workflows
- Components: Trigger, Action, Conditions, Loops, Branching

Types of Automations

- Personal automation (e.g., scheduling meetings)
- Business process automation (e.g., lead management)
- IT automation (e.g., infrastructure monitoring)
- Marketing automation (e.g., email campaigns)

Real-Life Use Cases

- Automate sending welcome emails after form submission
- Auto-posting content to social media
- Syncing Google Sheets with CRMs
- Daily reminders and task notifications

Workflow Examples

Lead Capture & Follow-up

Trigger: New form submission (Typeform/Google Form)

Workflow:

- Extract name, email, company.
- Enrich company info via API.
- Check company size.
- Small → Send a welcome email.
- Large → Add to CRM (HubSpot) and notify sales via Slack.
- Spam → Ignore.

Workflow Examples

eCommerce Order Processing

Trigger: New order in Shopify

Workflow:

- Update inventory in Google Sheets.
- Generate invoice (PDF & Email).
- Send shipping request to Shiprocket API.
- Notify customer on WhatsApp (via Twilio).

Workflow Examples

Student Onboarding

Trigger: Student registers via website form

Workflow:

- Add to Google Sheet (student database)
- Send welcome email with course login info
- Add to onboarding email sequence (Mailchimp)
- Schedule an orientation event in Google Calendar

Implementing Automations

Mapping

Always map out the automation process before building it

Implementing Automations

Benefits of Mapping Out a Process

- **Understanding:** Ensures you understand the task and its predictable actions
- **Visibility (Tools):** Provides clarity on the different tools, apps, or actions that will be used
- **Feasibility:** Helps assess if the automation is actually possible, preventing wasted time on impossible tasks
- **Workload Estimation:** Assists in estimating how long it will take to automate the task
- **Identify Human Intervention:** Shows where human intervention might still be necessary if the task cannot be fully predetermined

Implementing Automations

How to Map an Automation

- Create a flowchart

Steps:

- List Parts as Blocks:
 - List every different part of the process as an individual block
- Link with Arrows:
 - Work left to right, linking these blocks with arrows to show how they interact

Implementing Automations

Mapping is especially valuable for complicated automations; if something is unclear during mapping, clarify it before building to avoid issues later

Ex: 1 – Job Application Filter

A company is receiving job applications through an online form. Your task is to design a logic map (flowchart) that automates the screening and response process. The workflow should begin with a trigger when a new application is submitted. The system should then check the applicant's experience level and listed skills. Based on this evaluation, the logic should branch into three outcomes:

- If the applicant does not meet the criteria, send an automatic rejection email.
- If the applicant is a potential fit, add their details to the company's CRM system and send a notification to the HR team.
- If the applicant is considered a top-tier candidate, immediately notify the CEO via Slack for quick follow-up.

Draw this flowchart showing all decision points and actions clearly, and label each step logically to represent the full process.

Ex: 1 – Job Application Filter

Scenario:

A company receives job applications through a form.

Task:

- Create a logic map that:
 - Triggers from a new application
 - Checks for experience level and skills
 - If not a fit → send rejection email
 - If potential fit → add to CRM + notify HR
 - If top-tier → notify CEO via Slack

Ex: 2 – Order Management Automation

In this scenario, an online store wants to automate its order processing workflow. Begin by designing a flowchart where the process is triggered automatically when a new order is placed through the store's system. The workflow should first check the inventory to determine if the ordered item is in stock. If the item is available, the system should send a confirmation email to the customer and forward a shipping request to the logistics team or shipping API. However, if the item is out of stock, the workflow should instead send an “Out of Stock” notification email to the customer, possibly including estimated restock time or alternative options. Regardless of the stock status, the final step in the workflow should be to log the order details into a spreadsheet tool like Excel or Airtable for record-keeping and future analysis. Your flowchart should clearly show each condition, decision point, and action taken in the process.

Ex: 2 – Order Management Automation

Scenario:

An online store receives a new order.

Task:

- Map a workflow that:
 - Triggers when a new order is placed
 - Checks if the item is in stock
 - If yes → Send confirmation email and shipping request
 - If no → Send “Out of Stock” email to customer
 - Log the order details to an Excel sheet or Airtable



भारतीय प्रौद्योगिकी
संस्थान जम्मू
INDIAN INSTITUTE OF
TECHNOLOGY JAMMU



API

&

Webhooks

By

Parmveer Nandal

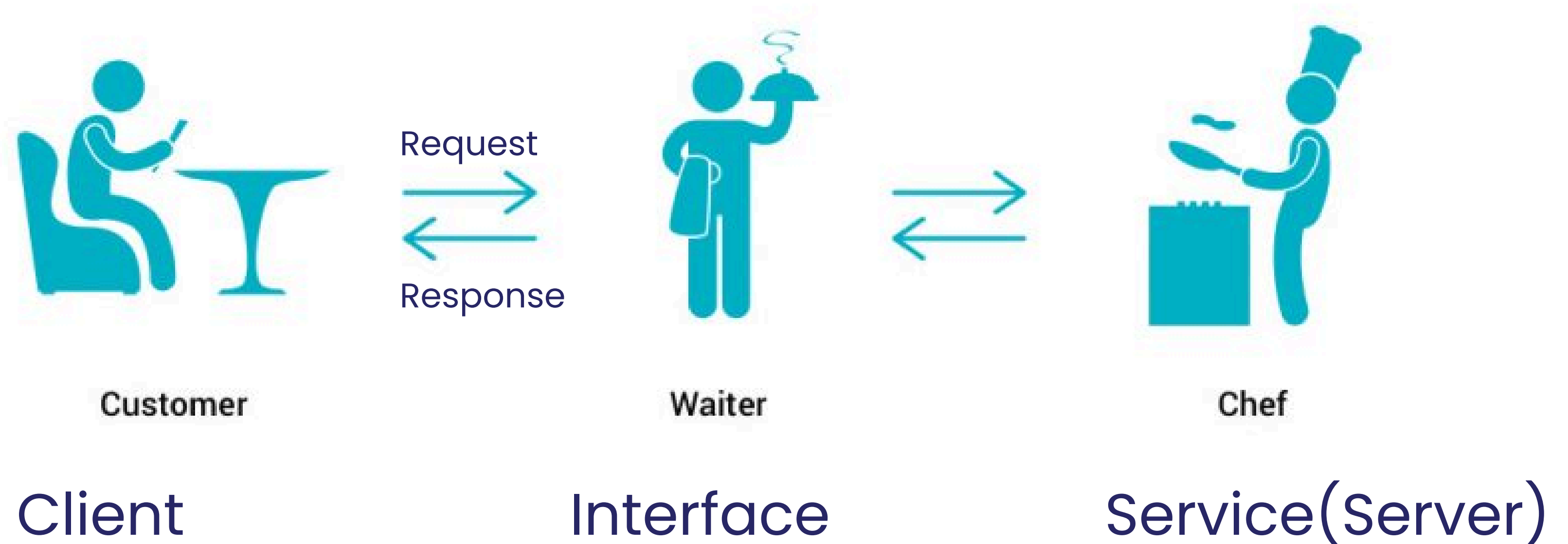
API

(Application Programming Interface)

An API exposes a service and developers write programs to consume it

API

(Application Programming Interface)



Why do we need APIs?

- **Connect Different Apps**

- APIs help one app use features or data from another app.
- Example: A travel app like MakeMyTrip uses flight data from airlines via their APIs.

- **Reuse Existing Functionality**

- You don't have to build everything from scratch — use APIs to plug into powerful services.
- Example: Use OpenAI's API to add AI features like text generation to your app.

Why do we need APIs?

- **Automate Workflows**

- APIs let tools like n8n automate actions between platforms.
- Example: When a user fills a form on your website, the data can automatically go to Google Sheets, a CRM, and Slack — thanks to APIs.

- **Secure and Standardized Data Exchange**

- APIs define rules for how apps share data securely and consistently.

How does an API work?

- **Documentation:**

- **An API uses documentation, which explains how the API works**
- In the restaurant analogy, the documentation is like the menu

- **Terminology:**

- Request: Something sent through the interface to the application
- Response: Something the application uses the interface to send back
- Client: You (the one making the request)
- Server: The application and its interface

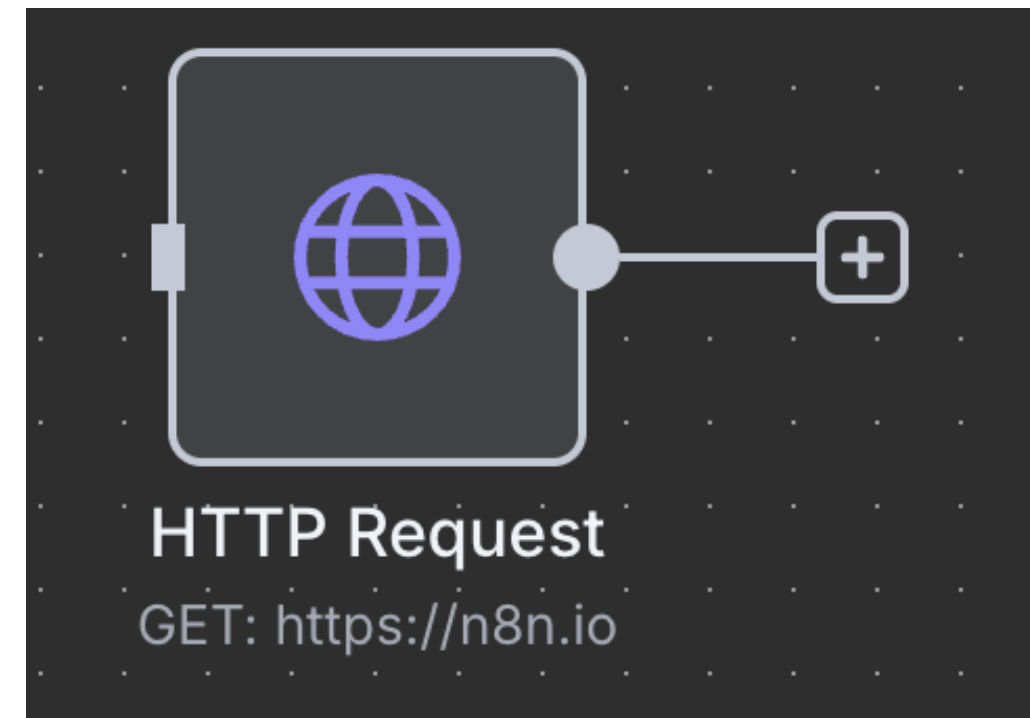
APIs in workflows

- Most workflow tools (like n8n) use APIs behind the scenes to:
 - Fetch data (e.g., Google Sheets)
 - Post updates (e.g., Twitter)
 - Send messages (e.g., Slack or Telegram)
- You can connect multiple APIs in a single workflow.

Component of a Request

4 components of an HTTP Request

- URL
- Method
- Header
- Body



Component of a Request

- **URL (Endpoint)**

- `https://www.google.com:443/search?q="Automation"`
- The address where the API is located.
- **Example:**
 - `https://api.openweathermap.org/data/2.5/weather`

- **Method:**

- Tells the API what action to perform.
- Common methods:
 - GET – Fetch data
 - POST – Send new data
 - PUT – Update data
 - DELETE – Remove data

Component of a Request

- **Header**

- Provides more detail or context for a given request
- Additional information (like authentication, data type, device type)
- **Example:** When browsing the internet, your browser sends a header indicating if you're on a computer or laptop

Authorization: Bearer YOUR_API_KEY

Content-Type: application/json

-

Component of a Request

- **Body:**

- Optional: Mainly exists for POST/PUT requests to send data.
- Usually in JSON format.
- Content: Contains the information you want to send to the server or application

- Example

```
{  
  "city": "Delhi",  
  "units": "metric"  
}
```

Ex: Weather API Request

Goal: Get the weather for Delhi

GET https://api.openweathermap.org/data/2.5/weather?q=Delhi&appid=YOUR_API_KEY

- Endpoint: <https://api.openweathermap.org/data/2.5/weather>
- Method: GET
- Query Params (in URL):
 - q=Delhi (City)
 - appid=YOUR_API_KEY (API key for access)
- Headers: (optional for GET)
- Body: Not needed (GET request)

Ex: OpenAI API

Goal: Ask OpenAI: "What are the benefits of automation?" and receive a natural language response.

- Endpoint: <https://api.openai.com/v1/chat/completions>
- Method: POST
- Headers: (optional for GET)
 - Authorization: Bearer YOUR_API_KEY
 - Content-Type: application/json
- Body: Not needed (GET request)

Ex: OpenAI API

- Body (Payload)

```
{  
  "model": "gpt-3.5-turbo",  
  "messages": [  
    {  
      "role": "user",  
      "content": "What are the benefits of automation?"  
    }  
  ],  
  "temperature": 0.7  
}
```

Ex: OpenAI API

- Example Response

```
{  
  "choices": [  
    {  
      "message": {  
        "role": "assistant",  
        "content": "Automation helps save time, reduce errors, and improve  
efficiency by allowing repetitive tasks to be handled by machines or  
software."  
      }  
    }  
  ]  
}
```

Credentials/Authentication

Purpose of Credentials:

- To inform the application that you are authorized to make a specific request
- Prevents unauthorized access (e.g., anyone reading or updating your Google Sheets)
- Most APIs require authentication, but some do not.

Common Authentication Methods:

- **Query Parameters:** ?API_key=YOUR_API_KEY4.
- **Header Authentication:** Authorization: Bearer YOUR_API_KEY4.
- **OAuth:** Used when you click "sign in with Google" and a window opens for you to sign in with your Google account

Component of a Response

Once a request is sent, the application will respond

3 main components of an HTTP Response

- Status Code

- Header

- Body

Status Codes

A three-digit number indicating whether the request was successful or unsuccessful

Common Status Codes:

- **200 (OK)**: Standard successful response; the request executed well
 - *General Rule*: If it starts with a '2', it was successful
- **401 (Unauthorized)**: Usually an authentication problem; the request lacked necessary authentication information. Means you need to check your credentials or access
 - *General Rule*: If it starts with a '4', you made an error and need to modify your request
- **404 (Not Found)**: Usually a URL problem; the resource (page/data) is not at the indicated URL
- **500 (Internal Server Error)**: Indicates a server error, not your fault
 - *General Rule*: If it starts with a '5', it was the server's error; most often, just try again later

Header & Body

- **Header (Response)**

- Similar to the request header, it provides more context or detail from the application.
- **Common Information:** Content length, content type, or content expiration

- **Body (Response):**

- The actual data being returned by the application.
- Formats: Can be HTML (for web browsing), JSON, or other forms of data (e.g., binary data)

What are Webhooks?

Webhooks are sometimes called "reverse APIs"

Analogy: Imagine you're waiting for food deliver at home

- Option 1 (Polling): You could go and check the door every few minutes to see if he has arrived
- Option 2 (Webhook): You could wait for the doorbell to ring, which indicates he has arrived

Webhook Function: The doorbell in this analogy is a webhook; it indicates that something you were waiting for has happened

Webhooks vs. Polling

(Payment Example)

Scenario: You need to know every time a new payment is made in your account

Polling:

- Making a new API request to Stripe every few minutes asking, "Is there a new payment?"
- If yes, perform actions; if no, wait and keep asking

Webhook:

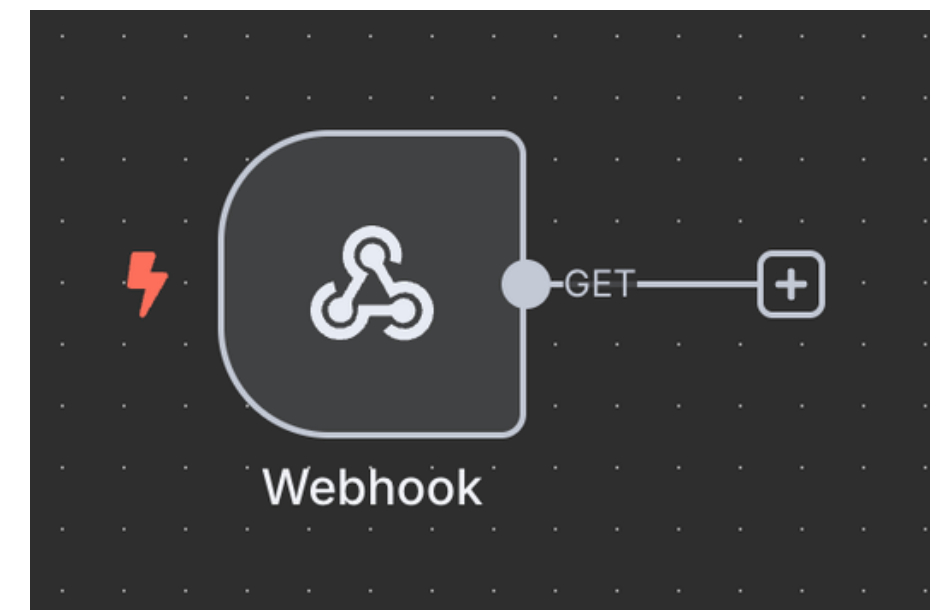
- Setting up a webhook so that every time a new payment is made in Stripe, a webhook is sent to you synchronously with the payment information
- More efficient for waiting for something to happen or for information from an application or service

Setting up a Webhook

Requirement: To set up a webhook, you need a URL

- This URL is the location where the information will be sent

- **n8n Usage:** You can use a tool like n8n and its webhook node to receive, manage, and deal with this information, triggering a workflow





भारतीय प्रौद्योगिकी
संस्थान जम्मू
INDIAN INSTITUTE OF
TECHNOLOGY JAMMU



Automations & Workflows

with



n8n

By

Parmveer Nandal

N8N Nodes

The node is "the atom of n8n".

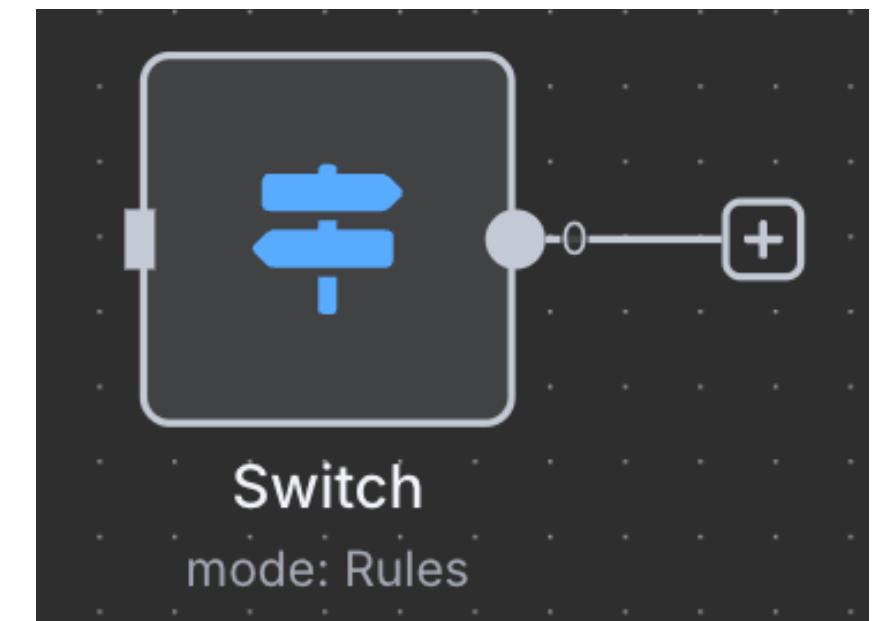
Every workflow is built by connecting consecutive nodes

Three Main Categories of Nodes:

1. **Entry Points:** These are your triggers
2. **Functions:** These allow you to transform, filter, or format data
3. **Exit Points:** These are your apps or applications

Main Categories of Nodes:

- Triggers, Actions in app, Data transformation, Flow, Files, Advanced nodes



Building Workflows:

- Canvas
- Adding Nodes
- Node Actions
 - Execute the Node
 - Additional Actions
 - Node Parameters
 - Node Setting

Credentials

- Located at the top of the parameters for each app node.
- This is how n8n authenticates to different applications and services.
- Credentials are saved at an instance level for efficiency and security, allowing sharing while preventing unauthorized access



Input & Output Data Views

Data Flow:

To the left of the node parameters, you have input data, and to the right, output data. This makes it easy to understand what data is being read by the node and its associated output

Three Main Views for Data:

- **Table View:** Displays data with different columns and values. Very useful when reading Google Sheets.
- **JSON View:** Shows the output data as key-value pairs.
- **Schema View:** Displays all the different keys from the input and examples of corresponding values

Data Structures

JSON (JavaScript Object Notation)

- A common digital data storage method.
- Written between curly braces {}
- Composed of key-value pairs, separated by commas
- **Embedded JSON:** A value of a key can be another JSON, used to organize complex data
 - Example: {"location": {"country": "USA", "city": "New York"}} where location is a key whose value is another JSON containing country and city information

Data Structures

JSON (JavaScript Object Notation)

- **Embedded JSON:** A value of a key can be another JSON, used to organize complex data
 - Example: `{"location": {"country": "USA", "city": "New York"}}` where location is a key whose value is another JSON containing country and city information
- **Accessing Data:** Use dot notation (.)
 - `'$json.first_name'` accesses the value of the `'first_name'` key
 - For embedded JSON: `$json.location.country` accesses the country value within the location JSON

Data Structures

Lists:

- A collection of objects
- Can contain objects of the same or different types
- Written between square brackets [], separated by commas
- [1, 2, d, b, same]
- Can be a list of any type of objects, including a list of JSONs
 - *Example:* [{"first_name": "Silvi", "last_name": "Sharma", "email": "..."}, {"first_name": "Raj", "last_name": "Mishra", "email": "..."}]

Correspondence Between JSON and Tables

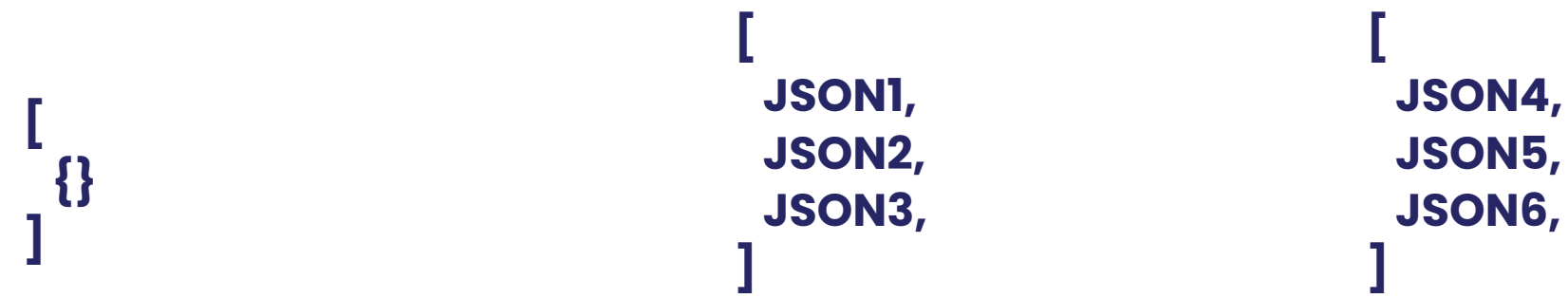
- **One JSON is equivalent to one row in a table**
- The keys of the JSON are the headers (columns) of those rows
- A list of JSONs is equivalent to a table with multiple rows
 - *Example:* A list of three JSONs corresponds to a table with three rows, each containing the values of the corresponding JSON
- In n8n, these JSONs in a list are called Items
- Nodes use items (plural) as both inputs and outputs
- These are the only accepted data formats for node inputs and outputs
- An empty output is represented as a list with an empty JSON: [{}]

How Nodes Use Items During Execution

- Each node executes once per item in the input data (some exceptions)
- *Example:* A Date & Time node processing a list of items with different dates will read each item, format its date, and return it as an individual item

Execution Schema

Output



Node



What happens

Step 1: Start Workflow: Clicking "execute workflow"

The "execute workflow" node returns an empty JSON [{}] in a list, allowing the next node to execute once

Step 2: Read Data (e.g., Google Sheets node):

- The Google Sheets node executes once
- It reads, for example, three items (rows) from a Google Sheet
- It returns these three items as JSONs (e.g., JSON 1, JSON 2, JSON 3), each linked to a corresponding row

Step 3: Filter Items (e.g., Filter node):

- The Filter node checks each item against a specific filter condition
- If only one item satisfies the condition, it outputs only that one item

Expressions

- Expressions are used to configure nodes based on input data
- They are created by dragging a key from the table JSON or schema view into a node's configuration
- Expressions for each item return the associated value to that key
- The n8n interface displays an example value from the first item
 - *Example:* Dragging the first_name key into a filter condition to allow only "Silvi"
 - The Expression Editor would show `$json.first_name` for item one is equal to "Silvi"
- Nodes execute once per item, so the expression's value dynamically updates for each item processed

“Anything between two curly braces `{ }` is considered an expression”

Use Case for Expressions

- **Item Variables:** Directly access values like `$json.first_name`, `$json.last_name`, or `$json.email`
- **JavaScript:** Apply built-in JavaScript functions or methods to item values within curly braces

Use Case for Expressions

- **Combine Expressions with Plain Text**

- Example for a Slack node message: `{{ $json.first_name }}`
`{{ $json.last_name }}` (`{{ $json.email }}`) just signed up for Course.
- This would produce a different message for each input item, such as "Silvi Sharma (silvi.sharma@gmail.com) just signed up for Course" for the first item
- *Practical example in the Edit Fields node:* Creating a full_name field using `$json.first_name` and `$json.last_name`
- Adding JavaScript to transform a value, such as turning the last name to uppercase: `{{ $json.first_name }}` `{{ $json.last_name.toUpperCase() }}` which would result in "Paul HARRIS"

Core Workflow Concepts – The Canvas and Workflow Elements

- **Main Workflow Elements**

- **Workflow Menu:** Located at the top left
- **Workflow Name and Tags:** Displayed up top
- **Workflow Activation Settings:** Top right corner
- **Version History and Workflow Specific Settings:** Also accessible in the top right corner
- **Nodes:** The main building blocks in the middle of the canvas
- **Zoom Settings:** Located at the bottom left

-

Core Workflow Concepts – The Canvas and Workflow Elements

- **Workflow Menu Details**

- Shows all workflows of an n8n instance
- Information like tags and owners, which can be filtered
- "My workflows" option to see workflows owned by the current user
- New workflows assign the creator as the default owner

- **Workflow Settings Details**

- Workflow accessibility
- Error workflow configuration
- Settings for how workflow executions are saved

Triggers

- Every workflow starts with a trigger
- Recognizable by having only an output branch
- Features an orange lightning icon next to them
- **They are used to launch an activated workflow**

Node Navigation

- Double-clicking a node shows the nodes before and after it
- Small icons are available to navigate through the workflow
- Example: Double-clicking a Filter node shows the Google Sheets node to the left and the Edit Fields node to the right

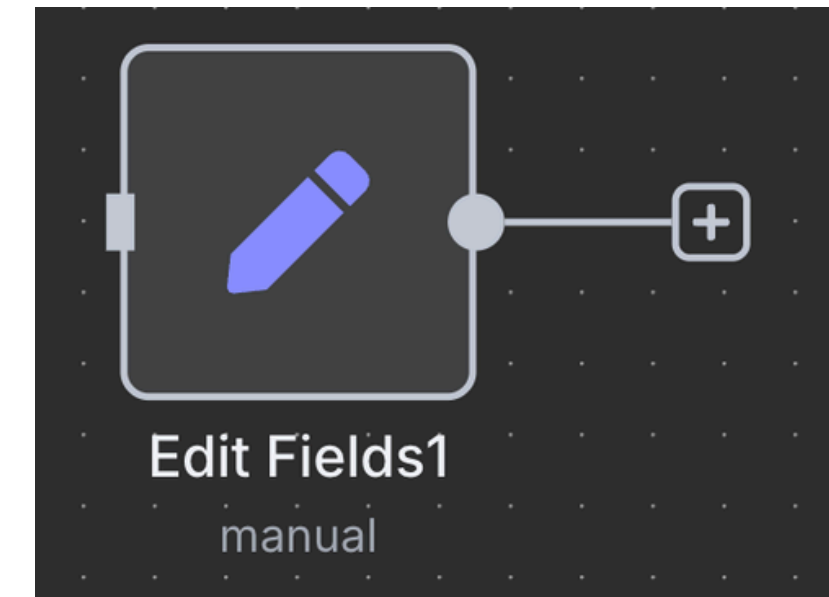
Branching: Creating Different Paths

- **Branching Method 1:** Node with Multiple Output Options
 - Description: "Use a node that has multiple output options"
 - Key Concept: Each item will follow only one of the multiple paths
- **Branching Method 2:** Dragging Multiple Output Lines from a Single Node
 - Description: "Dragging two or more output lines from a single node"
 - Key Concept: Every single item will follow every path and will be duplicated by the number of paths

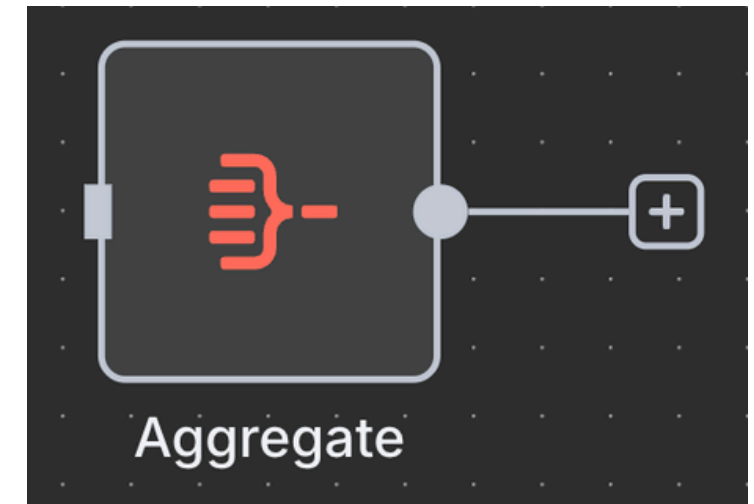
Some Useful
N8N Nodes

Edit Fields

- **Purpose:** Managing data in your items
 - Cleaning up data
 - Adding, formatting, or reducing any data within items
 - Ensures cleaner data for later workflow steps
- **Key Setting: "Include in Output"**
 - "Keep only" the fields being set
 - "Include all of the input fields"
- **Best Practice:** Always recommend using Edit Fields to keep only the useful data to avoid complex items with many unused fields



Aggregate Node



- **Purpose:** Aggregates data across all items
 - Takes multiple input items and combines specific fields into a single output item
- **Functionality**
 - Example: Two input items (each with an email) can be aggregated to one output item containing all emails
- **Related Nodes:**
- Remove Duplicates, Limit, Summarize, Split Out

Webhook Node



- **Purpose:** Listens for incoming webhooks to trigger a workflow
 - Takes multiple input items and combines specific fields into a single output item
- **Key Features:**
 - URL Attribution: When added, it is attributed a "Test URL" and a "Production URL"
 - Modifiability: URLs can be modified
 - **Execution:**
 - "Test URL" is used when testing the step/workflow
 - "Production URL" is used once the workflow is activated
- **Data Handling:** Executes the workflow with the received webhook data as the initial data